

Competitive Programming

Section 59 Editorial: ICPC Team Strategy

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Read before code: problem dossiers	3
1.2	Problem-specific proof map	3
1.3	Detailed solution guide	3
2	A. Team Assignment	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
3	B. Team Scoreboard	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Template Fibonacci	7
4.1	Restatement	7
4.2	Observations and algorithm	7
4.3	Correctness proof	7
4.4	Complexity	7
4.5	Full C++ solution	7
4.6	Full Python solution	7
5	D. Contest Schedule	9
5.1	Restatement and objectives	9
5.2	Greedy choice and trace	9
5.3	Algorithm	9
5.4	Correctness proof	10
5.5	Complexity, implementation notes, and tests	10
5.6	Full C++ solution	10
5.7	Full Python solution	11
6	What to carry forward	12

1 Editorial overview

1.1 Read before code: problem dossiers

A. Team Assignment. A mask assigns its selected tasks to one teammate; its complement goes to the other. Minimize $\max(\text{sum}, \text{total-sum})$ over all masks. Test a single task and a case with several optimal splits.

B. Team Scoreboard. The comparator is the contest rule: more solved, lower penalty, then name. Make it a tuple, not nested ad hoc comparisons; test all tie layers. Complexity $O(n \log n)$.

C. Template Fibonacci. Fast doubling stores $(F(k), F(k+1))$; identities produce $2k$ and $2k+1$, proving each halving step. Test 0, 1, and a huge index. Complexity $O(\log n)$.

1.2 Problem-specific proof map

A. Team Assignment. A bitmask represents exactly the tasks assigned to one person; its complement goes to the other. Enumerating masks covers all splits, so the smallest maximum load is optimal.

B. Team Scoreboard. Each row is reduced to the stated ranking key: higher solved count, lower penalty, then name. Sorting by that key implements every tie rule directly.

C. Template Fibonacci. Fast doubling maintains $(F(k), F(k+1))$; the doubling identities compute the pair for $2k$, and one addition advances to $2k+1$. Halving the index gives $O(\log n)$ time per query.

D. Contest Schedule. Keep shortest problem times to maximize the number solved before the limit. For a fixed selected set, shortest-processing-time order minimizes the sum of completion times, the ICPC penalty.

1.3 Detailed solution guide

A. Team Assignment Planning. The local task is a partition-style planning problem. Represent each problem's expected effort or value as an item, then use subset DP to find a balanced split. The invariant is that after processing the first i items, the reachable set contains exactly the sums achievable by choosing a subset of those items.

B. Scoreboard Simulation. ICPC ranking is deterministic once solved counts and penalties are known. Accumulate each team's solved count and penalty, then sort by the official order: more solved, less penalty, then name or id as the statement specifies. Keeping the comparator explicit avoids hidden tie bugs.

C. Template Recall. The Fibonacci/template exercise checks whether the team can deploy a known logarithmic routine under time pressure. Fast doubling keeps $(F(k), F(k+1))$; each

binary digit doubles the index and optionally advances by one. This is the same invariant used in the advanced mixed contest.

Common pitfalls. Strategy exercises should still have proofs. The proof is usually that the simulated rule, DP invariant, or template invariant is exactly the contest rule being asked for.

The problems model team operations with algorithms already taught.

- A balances implementation workload by subset partition.
- B applies contest ranking rules, useful for avoiding scoreboard interpretation mistakes.
- C is a trusted notebook drill: fast Fibonacci by doubling.

The programming content is deliberately familiar; the strategic lesson is to make known templates fast, shared, and reliable under ICPC constraints.

2 A. Team Assignment

2.1 Restatement

Split implementation workload between two people minimizing max time.

2.2 Observations and algorithm

Each bitmask assigns selected tasks to one teammate and all other tasks to the second. Compute both loads and minimize their maximum; trace a mask and its complement to see why it is redundant but still valid.

2.3 Correctness proof

Every two-way allocation maps to one mask, and every mask maps back to one allocation, so the minimum maximum load is optimal. $\square$

2.4 Complexity

The local bound permits $O(2^n)$ enumeration. Test a single task and equal total halves.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; a=data[1:]; best=10**9
    for mask in range(1<<n):
        s=sum(a[i] for i in range(n) if mask>>i&1); best=min(best,max(s,sum(a)-s))
    print(best)
if __name__=="__main__": main()
```

3 B. Team Scoreboard

3.1 Restatement

Rank teams by contest rules.

3.2 Observations and algorithm

Read a row as `(-solved, penalty, name)` and sort it. This is a simulation of the stated rule, not an algorithmic shortcut; test each tie-breaking layer.

3.3 Correctness proof

Lexicographic tuple order compares more solved first through negation, then lower penalty and finally name, exactly matching the contract. $\square$

3.4 Complexity

$O(n \log n)$ time and $O(n)$ memory. Keep names as strings; numeric conversion would lose the intended tie-break.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

3.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().splitlines()
    if not lines: return
    rows=[]
    for line in lines[1:]:
        name,sol,pen=line.split(); rows.append((-int(sol),int(pen),name))
    rows.sort(); print("\n".join(name for _,_,name in rows))
if __name__=="__main__": main()
```

4 C. Template Fibonacci

4.1 Restatement

Fast recurrence query drill for the team notebook.

4.2 Observations and algorithm

Maintain $(F(k), F(k+1))$. The doubling identities compute the pair for $2k$; when the next bit is one, advance once. Trace $k=5$ from the binary digits.

4.3 Correctness proof

The identities preserve adjacent Fibonacci values, and halving the index at each recursion/bit step reaches zero, proving the returned first component. $\square$

4.4 Complexity

Each query is $O(\log n)$ time and recursion depth. Test 0, 1, and a large index near the input limit.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; pair<long long,long long> fib(long long
```

4.6 Full Python solution

```
import sys
MOD=1_000_000_007
def fib(n):
    if n==0: return (0,1)
    a,b=fib(n//2)
    c=a*((2*b-a)%MOD)%MOD
    d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n%2 else (c,d)
```

```
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    print("\n".join(str(fib(x)[0]) for x in data[1:1+data[0]]))
if __name__=="__main__": main()
```

5 D. Contest Schedule

5.1 Restatement and objectives

One team works on one problem at a time. Problem i needs `time[i]` minutes; the contest lasts `limit` minutes. Choose and order problems so that as many as possible finish by the limit. Among schedules with the maximum number solved, minimize the total penalty: the sum of their completion times.

There can be 200,000 problems. The two objectives matter in that order: never sacrifice a solved problem merely to lower penalty.

5.2 Greedy choice and trace

Sort durations increasingly. Take them while their running sum remains within the limit. For durations `[3,1,2,2]` and limit 5, sorted durations are `[1,2,2,3]`. The first three fit, with completion times 1,3,5, hence:

```
solved = 3
penalty = 1 + 3 + 5 = 9
```

The fourth duration cannot fit. No set of four can fit because the sorted first four are already the four smallest durations. Among the three selected tasks, doing a longer one before a shorter one can only increase the completion-time sum.

5.3 Algorithm

```
sort all durations increasingly
elapsed = penalty = solved = 0
for duration in sorted durations:
    if elapsed + duration > limit:
        break
    elapsed += duration
    penalty += elapsed
    solved += 1
print solved, penalty
```

All durations are positive. Once the next sorted duration fails, every remaining duration is at least as large and also fails.

5.4 Correctness proof

For any k , the sum of the k smallest durations is no larger than the sum of any other k durations. Therefore if the greedy algorithm cannot fit the first k sorted tasks, no schedule can complete k tasks within the limit; and whenever it fits them, those tasks themselves form a feasible schedule. The number selected by the greedy algorithm is thus the maximum possible solved count.

Now fix any selected set. If two consecutive scheduled tasks have durations $a > b$, their contribution to the two completion times is $(P+a)+(P+a+b)$. Swapping them gives $(P+b)+(P+b+a)$, smaller by $a-b$, without changing any later completion time. Repeatedly swapping inversions gives nondecreasing duration order with minimum penalty. The greedy schedule uses exactly that order, so among maximum-count schedules it has minimum total penalty. QED.

5.5 Complexity, implementation notes, and tests

Sorting costs $O(n \log n)$ time and storage is $O(n)$. Use `long long` for elapsed time and penalty in C++; the penalty can exceed a 32-bit integer. Test no feasible problem, exact fit, repeated durations, and input ordered from longest to shortest. A frequent contest mistake is sorting only after selecting an arbitrary feasible set, which can lose the primary objective.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long limit;
    if (!(cin >> n >> limit)) return 0;
    vector<long long> durations(n);
    for (long long& duration : durations) cin >> duration;
    sort(durations.begin(), durations.end());

    long long elapsed = 0;
    long long penalty = 0;
    int solved = 0;
    for (long long duration : durations) {
        if (elapsed + duration > limit) break;
        elapsed += duration;
        penalty += elapsed;
        solved++;
    }
    cout << solved << ' ' << penalty << '\n';
    return 0;
}
```

5.7 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, limit = data[0], data[1]
    durations = sorted(data[2 : 2 + n])
    elapsed = 0
    penalty = 0
    solved = 0
    for duration in durations:
        if elapsed + duration > limit:
            break
        elapsed += duration
        penalty += elapsed
        solved += 1
    print(solved, penalty)

if __name__ == "__main__":
    main()
```

6 What to carry forward